PRACTICAL – 1

Aim: Distinct points (Point.java + Driver.java with main)
(a) Create class Point with private int x, y and a constructor.
(b) Override toString() to return "(x, y)".
(c) Override equals(Object) to be true when both x and y match, and hashCode() to agree
(e.g. Objects.hash(x, y)).
(d) In Driver.main: build a Point[] with some repeated coordinates.
(e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.
(f) Print "Distinct: N".
Expected output: 5 points with 2 repeats → "Distinct: 3".

Code Screenshot:

```
import java.util.Objects;

public class Pract3_1 {
    static class Point {
    private int x;
    private int y;

        Point(int x , int y){
            this.x = x;
            this.y = y;
        }

        @Override
        public String toString() {
            return "(" + this.x + ", " + this.y + ")";
        }

        @Override
        public boolean equals(Object obj) {
            if (this == obj) {
                return true;
            }
            if (obj == null || getClass() != obj.getClass()) {
                return false;
            }
```

```
        Point other = (Point) obj;
        return this.x == other.x && this.y == other.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.x, this.y);
    }
}

public static void main(String args[]){
    Point p[] = {
        new Point(1,2),
        new Point(2,3),
        new Point(4,5),
        new Point(1,3),
        new Point(6,7),
        new Point(8,9),
        new Point(2,7),
        new Point(4,3),
    };

    int count = 0;

    for(int i = 0 ; i < p.length ; i++){
        boolean check = false;

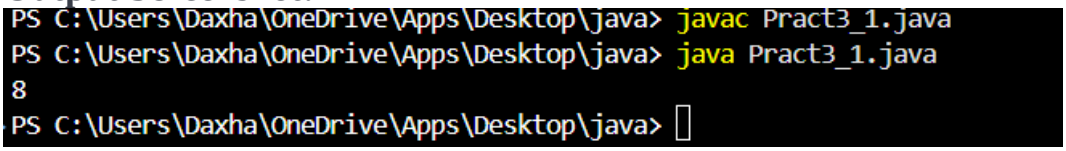
        for(int j = 0 ; j < i ; j++){
            if(p[i].equals(p[j])){
                check = true;
                break;
            }
        }

        if(!check){
            count++;
        }
    }

    System.out.println(count);
```

```
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create a class with private variables and a constructor.
- Learned how to override toString(), equals(), and hashCode() methods.
- Learned how to compare objects using the equals() method.
- Learned how to find and count distinct objects in an array by checking for duplicates.

Aim: Duplicate card check (Card.java + Driver.java)
(a) Create class Card with private String rank, suit and a constructor.
(b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").
(c) Override equals()/hashCode() by rank + suit.
(d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals()
to check it against the earlier cards.
(e) Print "Duplicate found: <card>" the first time a repeat appears.
Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spades".

Code Screenshot:

```
import java.util.*;

public class Pract3_2 {
static class Card {
private String rank;
private String suit;

Card(String rank, String suit) {
this.rank = rank;
this.suit = suit;
}

@Override
public String toString() {
return "(" + this.rank + ", " + this.suit + ")";
}

@Override
public boolean equals(Object obj) {
if (this == obj) {
return true;
}
if (obj == null || getClass() != obj.getClass()) {
return false;
}
Card other = (Card) obj;
return Objects.equals(this.rank, other.rank) && Objects.equals(this.suit,
other.suit);
```

```
}

@Override
public int hashCode() {
return Objects.hash(this.rank, this.suit);
}
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.println("Enter No of cards : ");
int n = sc.nextInt();
sc.nextLine();

Card c[] = new Card[n];
int count = 0;

for (int i = 0; i < n; i++) {
boolean check = false;
System.out.println("Enter rank for card " + (i + 1) + ": ");
String r = sc.nextLine();
System.out.println("Enter suit for card " + (i + 1) + ": ");
String s = sc.nextLine();
c[i] = new Card(r, s);

for (int j = 0; j < i; j++) {
if (c[i].equals(c[j])) {
check = true;
break;
}
}
if (!check) {
count++;
}
}
System.out.println("Total unique cards: " + count);
sc.close();
}
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create a Card class with private variables and a constructor.
- Learned how to override toString(), equals(), and hashCode() methods.
- Learned how to compare two Card objects to find duplicates.
- Learned how to check for duplicate cards while adding them to an array.
- Learned how to count unique cards and identify repeated cards.

Aim: (For Advanced Learners – home) Fractions (Fraction.java + Driver.java)

(a) Create class Fraction with private int num, den.

(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide

both by g.

(c) Override toString() to return num + "/" + den.

(d) Override equals()/hashCode() using the reduced num and den.

(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they

are all equal and print the same.

Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2).

Code Screenshot:

```
public class Pract3_3 {

public static void main(String[] args) {

Fraction first = new Fraction(1, 2);

Fraction second = new Fraction(2, 4);

Fraction third = new Fraction(3, 6);


System.out.println(first);

System.out.println(second);

System.out.println(third);

System.out.println("All equal: "

+ (first.equals(second) && second.equals(third) && first.equals(third)));
```

```
}
}

/** An immutable fraction stored in lowest terms. */
class Fraction {
private int num;
private int den;

public Fraction(int num, int den) {
if (den == 0) {
throw new IllegalArgumentException("Denominator cannot be zero.");
}

long numerator = num;
long denominator = den;
long g = gcd(numerator, denominator);
numerator /= g;
denominator /= g;

// Keep the denominator positive, so equal values have one representation.
if (denominator < 0) {
numerator = -numerator;
denominator = -denominator;
```

```
}

this.num = Math.toIntExact(numerator);
this.den = Math.toIntExact(denominator);
}

private static long gcd(long a, long b) {
a = Math.abs(a);
b = Math.abs(b);
while (b != 0) {
long remainder = a % b;
a = b;
b = remainder;
}
return a;
}

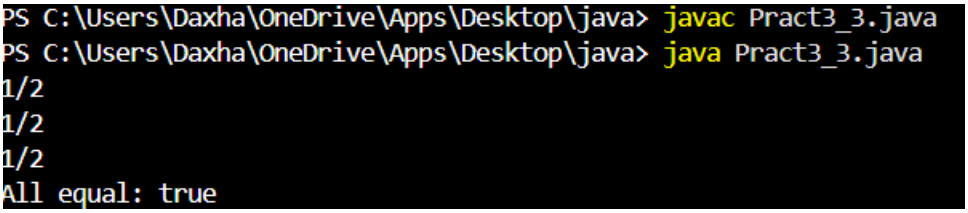
@Override
public String toString() {
return num + "/" + den;
}

@Override
```

```
public boolean equals(Object other) {
if (this == other) {
return true;
}
if (!(other instanceof Fraction)) {
return false;
}
Fraction fraction = (Fraction) other;
return num == fraction.num && den == fraction.den;
}

@Override
public int hashCode() {
return 31 * num + den;
}
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create a Fraction class with a constructor and private variables.
- Learned how to reduce fractions to their lowest terms using the GCD method.
- Learned how to override toString(), equals(), and hashCode() methods.
- Learned how to compare equivalent fractions, such as 1/2, 2/4, and 3/6.
- Learned how to represent equal fractions in the same simplified form.